GIA: LLM-Enabled Generative Intent Abstraction to Enhance Adaptability for Intent-Driven Networks

Shiwen Kou, Chungang Yang, *Senior Member, IEEE*, and Mohan Gurusamy, *Senior Member, IEEE*

***Abstract*—Conventional intent-driven networks (IDNs) rely on static templates to translate declarative intents into imperative network configurations. However, the dynamic nature of network environments and evolving user requirements pose significant challenges to these inflexible approaches. With the rapid advancement of generative artificial intelligence (GenAI), there have been approaches in IDNs that have attempted to leverage Large Language Models (LLMs) for intent translation. Despite their promising preliminaries, these efforts generally employ few-shot learning techniques, which are often inadequate in scenarios where new or complex intents surpass the capabilities of the limited training examples. To overcome these limitations, we introduce a Generative Intent Abstraction (GIA) framework, which leverages LLMs with knowledge-enhanced prompting techniques to dynamically generate adaptable intent graphs by compelling the LLMs to write Python codes. Furthermore, we reformulate the intent negotiation into a subgraph isomorphism problem (SIP), ensuring the generated graphs continuously comply with the network capabilities. Experimental results on the network intent dataset from easy and medium levels to hard and previously unseen levels demonstrate the effectiveness and adaptability of the proposed GIA framework. It significantly outperforms traditional methods without knowledge augmentation, achieving more than 30% higher F1 score for medium-level intents and a significant 180% increase for previously unseen intents.**

***Index Terms*—Intent translation, graph generation, intent-driven networks, intent negotiation, large language models.**

I. Introduction

OVER the past decade, software-defined networking (SDN) has transformed network management by simplifying configurations compared to conventional methods. However, despite these advancements, frequent human intervention is still required for network management [1]. Emerging as a novel networking paradigm, intent-driven networks (IDNs) aim to reduce manual maintenance by automatically translating service intents into specific network configurations [2], [3]. This enables users to focus on articulating their desired network objectives in natural language, termed "intents," rather than complex technical details.

Previous efforts in IDNs utilize structured intent templates to translate high-level user intents into specific network configurations. This approach employs an intermediate *abstraction* that decomposes high-level user intents into device-centric configuration policies. Notably, user-defined intent templates, such as Network Intent LanguagE (NILE) [4], Structured Network Intent Language (SNIL) [5], and Language for Access control list Intents (LAI) [6], have gained wider attention for formalizing and representing network intents. IDN systems like Merlin [7], Janus [8], PGA [9], and Kinetic [10] effectively leverage these languages to represent and fulfill user intents.

However, continuously changing network environments and evolving user requirements present persistent challenges for these conventional, template-based IDN systems. For example, the initial version of NILE [4] provided only minimal operational support for intent definition, focusing primarily on service-chaining capabilities. Subsequently, the authors in [4] extended this version to include crucial features for network management in real-world environments. Similarly, SCRIBE [11] enhanced the NILE grammar to incorporate more sophisticated functions, thereby facilitating a more precise representation of user requirements. This illustrates the pressing need for a more flexible and adaptable intent abstraction that can dynamically meet evolving user demands.

In response to these demands, recent breakthroughs in generative artificial intelligence (GenAI) have shown substantial promise. GenAI offers the flexibility to adapt to changing network environments and user requirements due to its capabilities in natural language understanding, reasoning, and generating high-quality responses. Among the applications of GenAI, large language models (LLMs) are particularly relevant for processing and generating human-like text, making them well-suited for complex intent handling and policy reasoning. For instance, Emergence [12] proposed a policy-based approach with pre-defined finite state machines (FSMs) to fulfill intents. Similarly, the authors in [13] and [14] introduced a pipeline that progressively decomposes intents using three LLMs with few-shot training to separate, classify, decompose, validate, and implement the generated policy set. Despite their promising preliminaries, there are still several limitations for LLMs-based IDNs that may impede their effectiveness.

1) *Interpretation:* LLMs often function as "black boxes" [15], [16], which poses a major challenge in

Received 30 July 2024; revised 9 November 2024 and 8 January 2025; accepted 17 February 2025. Date of publication 25 February 2025; date of current version 9 April 2025. The authors gratefully acknowledge the financial support from the Innovation Capability Support Program of Shaanxi (Program No. 2024RS-CXTD-01) and National Key Laboratory of Multi-domain Data Collaborative Processing and Control(Program No. MDPC20240401). The associate editor coordinating the review of this article and approving it for publication was V. Christos. *(Corresponding author: Chungang Yang.)*

Shiwen Kou is with the State Key Laboratory on Integrated Services Networks, Xidian University, Xi'an 710071, China (e-mail: shiwenkou@gmail.com).

Chungang Yang is with the National Key Laboratory of Multi-Domain Data Collaborative Processing and Control, Xidian University, Xi'an, China, and also with the Hangzhou Institute of Technology, Xidian University, Hangzhou, China (e-mail: shiwenkou@gmail.com; guideyang2050@163.com).

Mohan Gurusamy is with the Communication and Networks Lab, Department of Electrical and Computer Engineering, National University of Singapore, Singapore 117583 (e-mail: gmohan@nus.edu.sg).

Digital Object Identifier 10.1109/TCCN.2025.3545589

2332-7731 © 2025 IEEE. All rights reserved, including rights for text and data mining, and training of artificial intelligence and similar technologies. Personal use is permitted, but republication/redistribution requires IEEE permission.
See https://www.ieee.org/publications/rights/index.html for more information.
Authorized licensed use limited to: Amrita Vishwa Vidyapeetham Chennai Campus. Downloaded on January 03,2026 at 12:35:01 UTC from IEEE Xplore. Restrictions apply.

domain-specific tasks. Their tendency to "hallucinate"—producing irrelevant or misleading answers—is particularly problematic in areas requiring high reliability [17], [18], [19]. Recent studies such as AlphaCode [20], Copilot [21], and [22] suggest that LLMs have shown exceptional capabilities in code generation tasks, which effectively mitigates the hallucinations. This is because LLMs trained on large code repositories tend to perform more reliably on code-related tasks, making fewer mistakes. Code outputs also improve transparency, allowing operators to trace and understand the LLM's decision-making process directly. However, relevant investigation into using this technique (compelling LLMs to generate code) to mitigate hallucinations remains underexplored in the context of IDNs.

2) *Adaptability:* Recent LLMs applications to IDNs have predominantly employed few-shot learning techniques to teach LLMs to perform tasks within a pipeline using only a small number of examples [12], [13], [14]. While these methods introduce constraints to enhance output accuracy, their effectiveness is typically limited to narrowly defined scenarios and fails to adapt to continuously evolving user requirements. A significant factor in this adaptability issue is the absence of a robust, universal intent abstraction that can directly interact with LLMs, bridging the gap between declarative intents and imperative configurations. Although research has indicated that LLMs perform well with graph-based structures [22], to the best of our knowledge, investigations into generative graph-structured intent abstraction for IDNs have not been carried out so far; this is one of the motivations of this work.

In this paper, we aim to address the limitations of existing LLM-based applications in IDNs, specifically their lack of adaptability. To overcome these challenges, we propose a novel generative intent abstraction (GIA) language designed to improve adaptability to evolving user requirements. This is achieved by code generation, which makes the intent translation more transparent. GIA employs a graph-based representation, where nodes represent intent targets, edges depict relationships, and attributes denote intent quality of service (QoS) requirements. Unlike conventional IDN approaches that rely on static, predefined templates to represent intents, the proposed GIA framework utilizes LLMs to directly generate and manage these intent graphs (called GIA graphs) through code generation. Fig. 1 illustrates how LLMs dynamically generate these GIA graphs, enhancing the adaptability of the intent representation. In summary, our paper makes the following novel contributions:

- We present a novel framework for IDNs that leverages LLMs to enhance the autonomy and flexibility of network management in dynamic environments with evolving user requirements.
- To translate high-level intents, we introduce a novel GIA language that leverages the capabilities of LLMs to dynamically generate intent graphs. Specifically, our approach uses knowledge-augmented prompting, which incorporates relevant network knowledge to guide the generation of graph codes. This technique greatly enhances the flexibility and adaptability of abstracting changing user intents.
- To negotiate network conditions and resolve conflicts within the GIA framework, we transform the conflict detection task into a constrained subgraph isomorphism problem. This approach ensures that the generated GIA graph continually aligns with real-time network states, thereby improving intent translation accuracy and feasibility in dynamic network environments.
- Extensive experiments were conducted to validate the effectiveness of our proposed GIA framework. The results demonstrate that our framework significantly reduces hallucination in code generation and enhances the adaptability of intent translation, outperforming non-knowledge-augmented methods by over 30% for the seen intents and 180% for the unseen intents in terms of the F1 score.

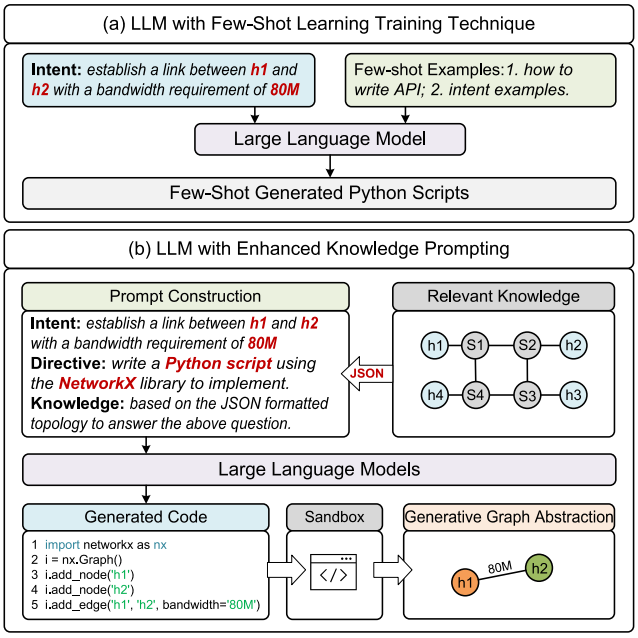


Fig. 1. Motivation for the proposed GIA framework. (a) Although few-shot training techniques are employed to prompt accurate responses and reduce hallucination from the LLM, it is impractical to provide the model on every conceivable intent type and API function. This challenge is further exacerbated by ongoing changes in network conditions and evolving user requirements. (b) The GIA framework retrieves the relevant network knowledge with regard to the specified intent and incorporates this data into the prompt. By compelling the LLMs to generate Python code using the NetworkX library, intents can be represented more efficiently and effectively in a zero-shot manner.

The remainder of the paper is structured as follows: Section II reviews related work on intent translation. Section III provides an overview of the GIA framework and its problem formulation. Section IV details the framework design for GIA. Section V discusses the evaluation results. Finally, Section VI concludes the paper.

II. Related Works

IDNs are designed to enhance network automation by enabling systems to comprehend and execute user-specified objectives. This section reviews prior works on IDNs, providing a detailed analysis of NLP-based and GenAI-based

Authorized licensed use limited to: Amrita Vishwa Vidyapeetham Chennai Campus. Downloaded on January 03,2026 at 12:35:01 UTC from IEEE Xplore. Restrictions apply.

IDN frameworks. We identify the limitations and challenges of these approaches, highlighting the necessity for our proposed GIA framework.

A. NLP-Enabled IDNs and Network Management

An intent is a set of operational goals specified in natural languages, describing how the network should behave. To ensure these intents can be interpreted and processed by IDN systems, an abstraction language is required to translate these intents into structured specifications. Specifically, these intents are represented through these abstraction languages (*i.e.,* predefined templates).

Among the various implementations of these abstraction languages, NILE [4], [23] is one of the most widely used because of its simplicity and close resemblance to natural language. NILE allows users to represent intents in a straightforward manner, making it accessible for individuals who may not have extensive technical expertise. For example, a user can specify a high-level intent like "ensure high bandwidth for Web services", which can then be represented in a more detailed and structured NILE format. Its design focuses on readability and ease of use, reducing the learning curve for network operators.

However, NILE's reliance on predefined grammar and static templates limits its adaptability. This inflexibility becomes problematic in dynamic network environments where user requirements continuously evolve and new intents emerge that are not covered by existing templates. For instance, SNIL [5] has to extend NILE's grammar to differentiate between query and network intents, providing more specific handling of different types of requests. Similarly, SCRIBE [11] needs to update NILE's syntax to more precisely describe forwarding behaviors in network address translation (NAT). Another notable example is LAI [6], which introduces natural primitives (*e.g.,* "check", "fix" and "modify") to simplify the management of Access Control List (ACL). Despite its effectiveness, LAI still encounters challenges when adapting to shifting scenarios or updated intent requirements, as current primitives may not be sufficient.

These examples illustrate the inherent limitations of static intent abstraction languages in adapting to the dynamic and evolving user requirements. This underscores the critical need for the IDNs framework to incorporate built-in mechanisms to automatically and proactively accommodate scenarios where predefined templates fall short.

B. GenAI-Enabled IDNs and Network Management

Recent advances in GenAI, particularly with transformer-based models like Generative Pretrained Transformers (GPTs), have revolutionized AI. Conventional NLP-based intent abstraction languages often struggle with processing large contexts and translating complex intents [1]. However, the capabilities of GPT-3.5 GPT-4 in contextual understanding and cognitive reasoning have introduced a new paradigm to IDNs with dynamic adaptability.

1) Few-Shot Prompting: LLMs have been shown to be effective few-shot learners [28], allowing them to perform new tasks with just a few training examples. As a result, recent studies have leveraged LLMs in IDNs using few-shot training techniques to translate high-level intents into network configurations with limited examples.

Among these approaches, AppleSeed [25] explores the capability of LLMs to translate network intents by utilizing an intermediate program abstraction. Specifically, they first predefine a set of program functions with names, parameters, and descriptions, but without implementations. Based on the specified network intents, they then guide the few-shot trained LLMs to complete these unimplemented functions by generating code. This approach allows network intents to be decomposed and fulfilled through a combination of the generated functions.

To decompose complex intents, Emergence [12] adopts a policy-based abstraction to translate complex intents into a tree of policies. Guided by FSMs, these decomposed policies are executed by hierarchical closed control loops. Furthermore, several studies employ the "chain of thoughts" technique to refine the application of LLMs in IDNs. For instance, the authors in [13], [14] further expand on Emergence by using a series of LLMs to ensure continuous intent fulfillment with intent assurance. The authors in [27] introduce customized LLMs as intelligent IDN agents. These agents methodically separate the intent fulfillment process into distinct phases, eventually refining declarative intents into abstraction language in JavaScript Object Notation (JSON) format. NetLM [26] employs LLMs to incrementally refine intents at different abstractions to address the ambiguity and incompleteness of declarative intents.

While the aforementioned approaches have shown greater effectiveness in IDNs, they may face challenges in generalizing across unseen scenarios due to limited (*i.e.,* few-shot) training examples. For example, predefined FSMs can restrict flexibility and adaptability, especially if these FSMs are not dynamically generated. Generating functional code seems a promising method to address the adaptability issues; however, predefined program function templates might also limit the system's ability to accommodate all potential network intents.

2) Knowledge-Enhanced Prompting: LLMs are able to perform zero-shot tasks, for example, generating responses based on the knowledge stored in pre-trained parameters, without additional training. However, since the knowledge in the pre-trained LLMs might be incomplete, incorrect, and outdated, they often generate factually incorrect answers due to their lack of intrinsic knowledge about specific tasks [29].

To improve their adaptability in domain-specific tasks, several studies suggest that incorporating factual knowledge is a promising approach to enhance its generalizing capabilities while mitigating hallucinations [16]. One notable example is KAPING [30], which retrieves relevant facts from knowledge graphs to supplement the prompts for LLMs. This approach improves the accuracy of LLM responses without requiring additional training. Another effective method is RHO [31], which aims to address hallucination by grounding dialogue responses in knowledge graphs through both local and global methods. To further enhance accuracy, RHO re-ranks generated responses by referencing relevant sub-graphs in the

Authorized licensed use limited to: Amrita Vishwa Vidyapeetham Chennai Campus. Downloaded on January 03,2026 at 12:35:01 UTC from IEEE Xplore. Restrictions apply.

TABLE I
COMPARISON OF INTENT ABSTRACTIONS IN IDN SYSTEMS

Category	System Names	Intent Parsing Mechanism	Intent Abstractions	Abstraction Adaptability	Code Generation (Anti-hallucination)
NLP-based	Lumi [24]	NER& Chatbox	NILE	Static Template (✗)	(-)
	SNIL [5]	NER& Chatbox	SNIL	Static Template (✗)	(-)
	Jinjing [6]	Structed Input	LAI	Static Template (✗)	(-)
	PGA [9]	Graphs	Policy Graph	Non-generated Graphs (✗)	(-)
	CHAT-IBN [25]	RASA	JSON	Static Template (✗)	(-)
GenAI-based	AppleSeed [26]	Few-shot Training	Function Program	Predefined Functions (✗)	Function Code (✓)
	Emergence [13]	Few-shot Training	Policy Trees	Predefined FSMs (✗)	(✗)
	LLM-based [14]	Few-shot Training	Policy Trees	Limited Examples (✗)	(✗)
	NetLM [27]	BiLSTM-CRF	Policy Trees	Predefined (✗)	(✗)
	LLM5G [28]	Chain-of-thought	JSON	Template (✗)	(✗)
	GIA (Ours)	Enhanced-prompting	Generated Graph	Generative Intent Abstraction (✓)	GIA Graph Code (✓)

KGs, ensuring the final outputs are more accurate and faithful. Similarly, KITLM [32] and StructGPT [33] retrieve relevant structure evidence and incorporate them into the LLMs while training and generating. These methods demonstrate the effectiveness of knowledge-enhanced prompting in zero-shot learning.

This paper aims to explore the scientific question of whether LLMs can be generalized to previously unseen intents and support dynamic environments. Based on the limitations illustrated in Table I, we introduce a novel generative intent abstraction to replace the predefined templates for intent translation.

III. ARCHITECTURE AND PROBLEM FORMULATION

In this section, we first provide a summary of the notation used in this work in Table II. We then introduce the basic system architecture and problem description of the proposed GIA framework.

A. System Architecture

As demonstrated in Fig. 2, the GIA framework accepts declarative intents in text form as input, and translates them into conflict-free graph abstractions, ensuring both accuracy and adaptability for network management. To function properly, the GIA framework comprises several key components, described as follows:

- *Prompt Constructor:* The prompt constructor generates prompts to effectively leverage the LLM's code generation capabilities. It enriches these prompts with network- and intent-relevant knowledge, denoted as $\mathcal{K}_{\text{selected}}$, aiming to help the LLM accurately translate high-level intents into structured GIA graphs.
- *Pretrained LLMs:* The pretrained LLMs generate responses according to the enhanced prompts. Given that they are trained on extensive corpora and massive code repositories, these LLMs are capable of interpreting complex and evolving network intents by writing codes.
- *Conflict Detection:* The conflict detection module identifies potential conflicts between generated policy graph G_p and deployed policy graph G_t. It ensures these generated G_p do not violate any constraints or previously deployed policies, thereby improving network stability and reliability.
- *Conflict Resolution:* The conflict resolution module addresses identified conflicts by regenerating policies graphs using network knowledge, such as topology $\mathcal{K}_{\text{topo}}$. If conflicts cannot be resolved locally, the module sends relevant error information back to the LLM for additional assistance.
- *Network Factual Knowledge:* This component leverages KGs to supply relevant external network knowledge, thereby enhancing the LLM's response relevance and precision. As a key module, it supports GIA graph generation and conflict resolution, enhancing the overall performance and reliability of the GIA framework.

TABLE II
SUMMARY OF NOTATIONS AND THEIR RESPECTIVE DESCRIPTIONS

Notation	Description
$\mathcal{I}$	Declarative intents specified in natural language
$\mathcal{C}$	Code snippets generated by LLM w.r.t $\mathcal{I}$
G_i	Generative intent abstraction graph w.r.t $\mathcal{I}$
G_p	Conflict unresolved policy abstraction graph
N_p	Set of G_p nodes
E_p	Set of G_p links
A_u	The amount of requested QoS by node $u \in G_p$
$A_{(u,u')}$	The amount of requested QoS by link (u, u')
G_t	Deployed policy graph that stores prior installed policies
N_t	Set of G_t nodes
E_t	Set of G_t links
A_v	The resource constrains supported by node $v \in G_t$
$A_{(v,v')}$	The resource constrains supported by link (v, v')
$\kappa(\cdot)$	An indicator function to evaluate the generated code
ϕ	The subisomorphism function
$\psi(\cdot)$	An indicator function to test if two graphs are subisomorphic
$\mathcal{D}$	Directive prompts that compel LLMs to generate code
$\mathcal{K}$	The network knowledge graphs used to supplement prompts
x''	The knowledge-enhanced prompt
G'_p	Conflict resolved policy abstraction graph

Authorized licensed use limited to: Amrita Vishwa Vidyapeetham Chennai Campus. Downloaded on January 03,2026 at 12:35:01 UTC from IEEE Xplore. Restrictions apply.

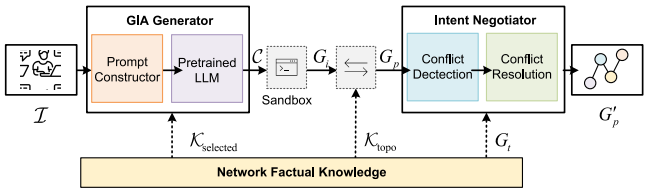


Fig. 2. Components of the proposed GIA framework. $\mathcal{I}$ is high-level network intents. $\mathcal{K}_{\text{selected}}$ represents the relevant network knowledge used to enrich the prompt. $\mathcal{C}$ is the generated code for $\mathcal{I}$. $\mathcal{K}_{\text{topo}}$ denotes the real-time topology that refines intent abstraction graph G_i into policy abstraction graph G_p. G_t is the policy graph storing previously deployed policies, and the output G'_p is the conflict-free policy abstraction graph.

B. Problem Formulation

To ensure the adaptability of the proposed GIA framework, we evaluate its performance in code generation and its effectiveness in resolving graph conflicts.

1) Problem Description for Code Generation: Consider a set of high-level intents $\mathcal{I} = \{I_1, I_2, \ldots, I_n\}$, where each intent $I \in \mathcal{I}$ is represented by a sequence of natural language tokens $I = \{d_1, d_2, \ldots, d_{|I|}\}$. By leveraging knowledge-enhanced prompts, we compel the LLM to generate corresponding code $\mathcal{C} = \{C_1, C_2, \ldots, C_n\}$ based on the intent tokens. The task of generating GIA graphs can be defined as:

$$f : \mathcal{I} \to \mathcal{C}, \tag{1}$$

where f is a mapping function.

To evaluate the success of this mapping, we define the compiler feedback function as:

$$\kappa(C_i) = \begin{cases} 1, \text{ if } \ C_i \text{ is successfully compiled,} \\ 0, \text{ otherwise,} \end{cases} \tag{2}$$

where $\kappa(C_i)$ is an indicator function that returns 1 if the code C_i is successfully compiled and 0 otherwise.

As a result, the objective of our problem formulation is to maximize the number of successfully compiled code instances. The optimization problem is expressed as:

$$\max_{f} \sum_{i=1}^{n} \kappa(C_i). \tag{3}$$

2) Problem Description for Intent Negotiation: In the intent negotiator, we aim to detect potential conflicts between the generated policy abstraction graph $G_p = (N_p, E_p)$ and the deployed policy graph $G_t = (N_t, E_t)$. Each node $u \in N_p$ and edge $(u, u') \in E_p$ have associated QoS requirements A_u and $A_{(u,u')}$, respectively. Similarly, each node $v \in N_t$ and edge $(v, v') \in E_t$ have monitored attributes A_v and $A_{(v,v')}$.

To identify conflicts, we need to ensure that nodes and edges in G_p can be matched to those in the deployed policy graph G_t without violating any constraints (*e.g.,* the current bandwidth of E_t should be greater than or equal to the specified bandwidth required by E_p). Therefore, we can transform the graph conflict detection problem into a subgraph isomorphism problem [34], where the task is to find a subgraph in G_t that matches G_p while satisfying the attribute constraints. Mathematically, this process can be expressed as finding an injective function:

$$\phi : N_p \to N_t, \tag{4}$$

such that each node in N_p maps to a node in N_t and each edge in E_p maps to an edge in E_t, *i.e.,* $\forall u \in N_p, \phi(u) \in N_t$, and $\forall (u, u') \in E_p, (\phi(u), \phi(u')) \in E_t$.

To evaluate whether G_p is subisomorphic to G_t, we define the indicator function $\psi(u)$ as:

$$\psi(u) = \begin{cases} 1, \text{ if } \ \phi(u) \in N_t \ \text{ and } \ \big(\phi(u), \phi(u')\big) \in E_t, \\ 0, \text{ otherwise,} \end{cases} \tag{5}$$

where $\forall u \in N_p$ and $\forall (\phi(u), \phi(u')) \in E_p$.

The objective is to maximize the number of matched nodes from N_p to N_t. The optimization problem can be formulated as:

$$\max_{\phi} \sum_{u \in N_p} \psi(u). \tag{6}$$

To comprehensively evaluate the adaptability of the GIA framework, we combine the objectives of both code generation and intent negotiation. The combined optimization problem can be formulated as:

$$\max_{f,\phi} \left(\alpha \sum_{i=1}^{n} \kappa(C_i) + \beta \sum_{u \in N_p} \psi(u) \right), \tag{7}$$

where α and β are weighting factors that balance the importance of successfully compiled code instances and the number of matched nodes, respectively.

IV. Detailed Design of the GIA Framework

This section introduces the proposed GIA framework, designed to dynamically translate high-level intents into structured graphs using LLMs. To address issues like hallucination and inflexibility, we incorporate a knowledge-enhanced prompt technique that improves code quality. Additionally, we introduce a graph-based intent negotiation mechanism to resolve conflicts between the GIA graph and the network state graph. These components work collectively to enhance flexibility and adaptability to meeting changing requirements in dynamic network environments. Fig. 3 illustrates the architecture of the proposed GIA framework.

A. Generative Intent Abstraction

While LLMs are adept at generating responses based on pre-trained knowledge, they may underperform in domain-specific scenarios due to outdated information or rapidly changing network conditions. To circumvent the resource-intensive process of continuous LLM fine-tuning, we leverage an enhanced prompting technique that incorporates network- and intent-relevant knowledge to translate user intents into graphical abstractions.

Authorized licensed use limited to: Amrita Vishwa Vidyapeetham Chennai Campus. Downloaded on January 03,2026 at 12:35:01 UTC from IEEE Xplore. Restrictions apply.

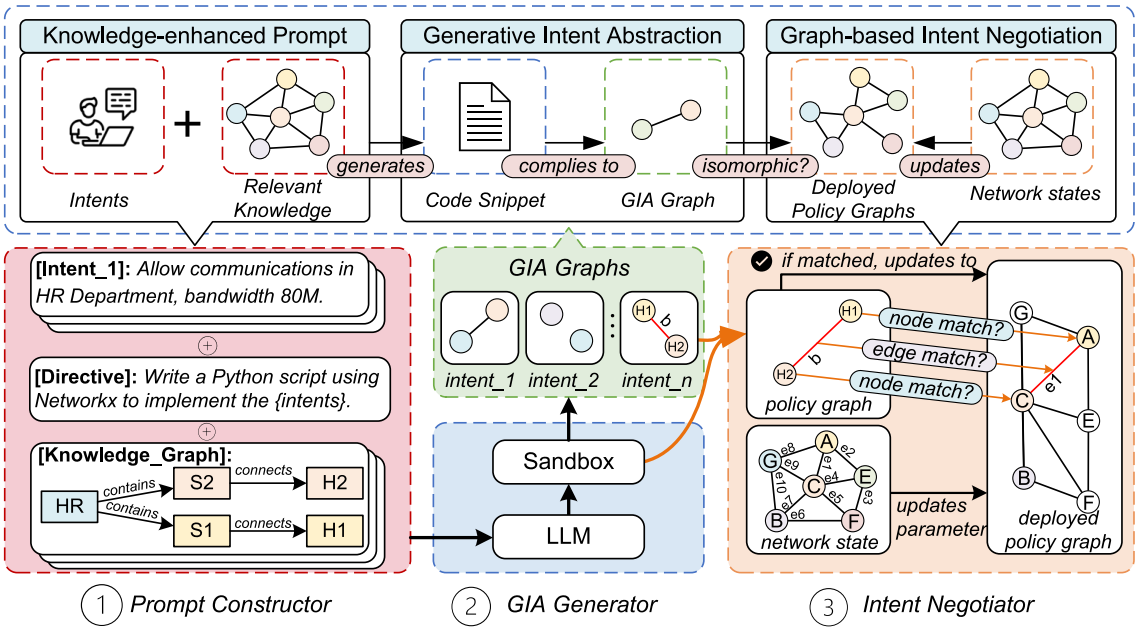


Fig. 3. The architecture of the proposed GIA framework.

1) Prompt Constructor: We define declarative intent as $I \in \mathcal{I}$ to guide the construction of our prompts. Fig. 3 (1) illustrates the three main components used in prompt construction.

Intent: Consider the user intent I: "Only employees in the financial department can access the financial servers". This intent specifies an access control policy to be implemented within the network, denoted initially as prompt x. Although LLMs understand this intent, they lack extrinsic knowledge for translating it into executable code or network configurations.

Directive: To facilitate LLMs in translating user intents, we enhance prompt x with a code-generation directive. This directive combines the user-specified intent with general program synthesis techniques, employing the NetworkX library [35]. This approach not only mitigates inherent limitations such as hallucinations [36] and arithmetic mistakes [37] but also transforms the abstraction intent translation task into a specific code generation task.

Formally, for any given basic prompt x, we append a directive $\mathcal{D}$ to create a structured prompt x', which is defined by:

$$x' = x \oplus \mathcal{D}, \tag{8}$$

where $\oplus$ denotes the operation of the appending directive to the existing basic prompt x. For example, the modified prompt x' could be Intent: "Only employees in the financial department can access the financial servers", Directive: "Write a Python script using the NetworkX library to implement the above {intent}."

Network Knowledge Graphs: Inspired by Baek et al. [30], we further augment prompt x' by incorporating relevant factual knowledge from an external knowledge base, denoted as $\mathcal{K}$. This knowledge database contains accurate and up-to-date factual triples such as topology, state information, and previously successfully deployed policies, which are represented as:

$$\mathcal{K} = (s, r, o), \tag{9}$$

where s and o denote subject and object entities, respectively, and r is the relation between them.

To integrate this knowledge, we select triples relevant to the user intent, defined as $\mathcal{K}_{\text{selected}}$ from $\mathcal{K}$ and append them to x', resulting in a further enhanced prompt x''. The formal definition of this process is expressed by the following transformation:

$$x'' = x' \oplus \mathcal{K}_{\text{selected}}, \tag{10}$$

where $\oplus$ denotes the operation of appending factual knowledge to the existing enhanced prompt x'.

The augmented prompt x'' helps LLMs generate responses that are not only accurate but also contextually relevant. The probability of generating an appropriate codes $\mathcal{C}$, given the knowledge-augmented prompt x'', is mathematically modeled as:

$$P\left(\mathcal{C} | x'', \mathcal{K}_{\text{selected}}\right), \tag{11}$$

where $\mathcal{C}$ represents the code generated by the LLMs.

2) GIA Generator: As illustrated in Fig. 3 (2), the GIA generator compiles and executes the code $\mathcal{C}$ within a secure execution sandbox. This process results in the creation of the GIA graph, formally defined as G_i.

Intent Graph: The GIA graph G_i contains vertices and edges that correspond to intent entities and their interconnections. Each edge has specific attributes related to network requirements, such as bandwidth. For instance, if the user's intent specifies a minimum bandwidth of '80m' between two hosts 'h1' and 'h2', the GIA generator creates nodes for 'h1' and 'h2' and connects them with an edge that carries this bandwidth attribute.

Authorized licensed use limited to: Amrita Vishwa Vidyapeetham Chennai Campus. Downloaded on January 03,2026 at 12:35:01 UTC from IEEE Xplore. Restrictions apply.

Policy Graph: After generating G_i, we further refine it into a policy graph, denoted as G_p. This policy graph enhances G_i by incorporating detailed network topology. The refinement from G_i to G_p is formally defined as:

$$G_p = g\big(G_i, \mathcal{K}_{\text{topo}}\big), \tag{12}$$

where $g(\cdot)$ represents the refinement function that integrates the network topology $\mathcal{K}_{\text{topo}}$ into the policy graph.

For example, if G_i describes 'h1' and 'h2' with a required bandwidth of 80M, the refinement function $g(\cdot)$ utilizes the network topology $\mathcal{K}_{\text{topo}}$ to introduce intermediate nodes 's1' and 's2'. Consequently, the policy graph G_p then delineates a detailed route: 'h1' $\rightarrow$ 's1' $\rightarrow$ 's2' $\rightarrow$ 'h2', ensuring each links have the bandwidth attributes specifications.

B. Graph-Based Intent Negotiation

After translating user intent into a network policy graph, the proposed GIA framework examines conflicts between new and existing policies and verifies compliance with the network's current capabilities. As illustrated in Fig. 3 (③), this subsection presents a general graph-based approach to model these challenges as a constrained subgraph isomorphism problem, enabling automatic conflict detection and intent resolution.

1) Constrained Subgraph Isomorphism Problem: To identify conflicts efficiently, our model uses Key performance indicators (KPIs) to describe the attributes of nodes and links. These attributes are essential for matching the generated policy graph G_p with the network's actual capabilities graph G_t. The attributes are uniformly structured for both nodes and links as:

$$\mathbf{A} = (a_1 : \text{val}_1, a_2 : \text{val}_2, \ldots, a_n : \text{val}_n), \tag{13}$$

where $\mathbf{A}$ represents a set of attributes, including both integer and boolean types. Integer parameters, such as latency and bandwidth, pertain to links, while boolean parameters relate to connectivity and operational intents, such as firewall settings, load balancing, and monitoring functions (*e.g.,* sFlow, DPI) for nodes.

Let $\mathbf{A}_u$ and $\mathbf{A}_v$ be two attribute vectors of KPIs for nodes $u \in N_p$ and $v \in N_t$, respectively, and similarly, $\mathbf{A}_{(u,u')}$ and $\mathbf{A}_{(v,v')}$ for their edges. We define the complete sets of attributes

$$A_p = \left(\bigcup_{u \in N_p} \mathbf{A}_u\right) \cup \left(\bigcup_{(u,u') \in E_p} \mathbf{A}_{(u,u')}\right) \tag{14}$$

and

$$A_t = \left(\bigcup_{t \in N_t} \mathbf{A}_v\right) \cup \left(\bigcup_{(v,v') \in E_t} \mathbf{A}_{(v,v')}\right), \tag{15}$$

corresponding to G_p and G_t, respectively. Because A_p may contain various QoS attributes declared by uses, our model enhances its applicability by ensuring attributes in A_p must be present in A_t. Specifically, if an attribute from A_p is absent in A_t, it is added with a wildcard '*' as its value, representing a placeholder. This update is mathematically represented by:

$$A_{t,new} = A_t \cup \big\{a | a \in A_p \backslash A_t \rightarrow \text{'*'}\big\}, \tag{16}$$

where $A_{t,new}$ denotes the updated attribute set in G_t.

To evaluate node compatibility, we assume node attributes in $\mathbf{A}_u$ and $\mathbf{A}_v$ are exclusively boolean. Therefore, we introduce a node feasibility match function to assess whether two nodes between G_p and G_t are compatible based on their attributes, which can be expressed as:

$$M_{\text{node}}(u,v) = \begin{cases} 1, \text{ if } \forall a_i \in \mathbf{A}_v, \\ \quad \text{either } \mathbf{A}_v[a_i] = \text{'*'} \text{ or } \mathbf{A}_v[a_i] = \mathbf{A}_u[a_i], \\ 0, \text{ otherwise,} \end{cases} \tag{17}$$

where the function returns 1 if every attribute in $\mathbf{A}_v$, the attribute value is either a wildcard '*' or matches the corresponding attribute value in $\mathbf{A}_u$. Otherwise, it returns 0.

Similarly, to evaluate link compatibility, we assume attributes in $\mathbf{A}_{(u,u')}$ and $\mathbf{A}_{(v,v')}$ can be either boolean (*e.g.,* connectivity) or integer (*e.g.,* bandwidth). Therefore, we introduce a link feasibility match function to assess whether two links are compatible based on their attributes, which is defined as:

$$M_{\text{edge}}\big((u,u'),(v,v')\big) = \begin{cases} 1, \text{ if } \forall a_i \in \mathbf{A}_{(v,v')}, \\ \quad \mathbf{A}_{(v,v'))}[a_i] \geq \mathbf{A}_{(u,u'))}[a_i], \\ 0, \text{ otherwise,} \end{cases} \tag{18}$$

where the function returns 1 if every attribute a_i in $\mathbf{A}(v,v')$ meets or exceeds the corresponding attribute value in $\mathbf{A}(u,u')$. Otherwise, it returns 0.

Given the feasibility, match functions $M_{\text{node}}(u,v)$ for nodes in Eq. (17) and $M_{\text{edge}}((u,u'),(v,v'))$ for edges in Eq. (18), we further refine Eq. (6) to maximize the sum of these matches across all nodes and edges in policy graph G_p when mapped to the network capability graph G_t. Thus, the objective function can be expressed as:

$$\max_{\phi} \Bigg[\sum_{u \in N_p} \sum_{v \in N_t} M_{\text{node}}(u,v) + \sum_{(u,u') \in E_p} \sum_{(v,v') \in E_t} M_{\text{edge}}\big((u,u'),(v,v')\big) \Bigg], \tag{19}$$

where N_p and E_p are the sets of nodes and edges in G_p, respectively, and N_t and E_t are the sets of nodes and edges in G_t, respectively. ϕ represents the optimization of nodes and edges from G_p to G_t that we seek to maximize.

2) Conflict Detection and Intent Negotiation: We propose an autonomous mechanism to simultaneously detect both policy conflicts and resource conflicts. We transform the problem of detecting conflicts between graphs into a subgraph isomorphism problem. To address the challenges of policy and resource detection within our GIA framework, we adopt the VF2 algorithm [38], a well-established method for solving subgraph isomorphism problems. This facilitates precise mapping of a policy graph G_p onto a network capabilities graph G_t. The VF2 algorithm operates by incrementally constructing a mapping between nodes of the two graphs, ensuring that each step adheres to the constraints of subgraph isomorphism.

Authorized licensed use limited to: Amrita Vishwa Vidyapeetham Chennai Campus. Downloaded on January 03,2026 at 12:35:01 UTC from IEEE Xplore. Restrictions apply.

Algorithm 1: Graph Conflict Detection

Data: $G_p = (N_p, E_p)$, $G_t = (N_t, E_t)$, $M_{\text{node}}(u, v)$, $M_{\text{link}}((u, u'), (v, v'))$
Result: Isomorphic mapping ϕ or failure

Function `Main`(G_p, G_t)**:**
  $\phi \leftarrow \varnothing$;
  $P_\phi \leftarrow \varnothing$;
  return `Isomorphic`(G_p, G_t, ϕ, P_ϕ);
Function `Isomorphic`(G_p, G_t, ϕ, P_ϕ)**:**
  if $|\phi| == |N_p|$ **then**
    return ϕ;
  else
    for *each* $u \in N_p$ *not in* ϕ **do**
      for *each* $v \in N_t$ *not in* ϕ **do**
        if `Match`(u, v, ϕ, P_ϕ) **then**
          $\phi \leftarrow \phi \cup \{(u, v)\}$;
          P_ϕ.push$((u, v))$;
          result $\leftarrow$ `Isomorphic`(G_p, G_t, ϕ, P_ϕ);
          if *result* $\neq$ *failure* **then**
            return *result*;
          $\phi \leftarrow \phi \setminus \{(u, v)\}$;
          P_ϕ.pop();
  return *failure*;
Function `Match`(u, v, ϕ, P_ϕ)**:**
  if $M_{node}(u, v) \neq 1$ **then**
    return *false*;
  for *each* $(u', v') \in P_\phi$ **do**
    if $\exists (u, u') \in E_p$ **then**
      if *not* $\exists (v, v') \in E_t$ *or* $M_{link}((u, u'), (v, v')) \neq 1$ **then**
        return *false*;
  return *true*;

This process involves evaluating potential pairings based on both structural connectivity and attribute compatibility. Alg. 1 details the conflict detection process.

Mapping Expansion: It initiates with an empty mapping $\phi = \varnothing$ and gradually extends it and build a partial solution ϕ by exploring and adding potential matches between nodes (u, v) and edges $((u, u'), (v, v'))$. This process utilizes a candidate set P_ϕ of pairs (u, v), which is defined as:

$$P_\phi = \{(u, v) | u \in \text{Neigh}(\phi), v \in \text{Neigh}(\phi), (u, v) \notin \phi\}, \quad (20)$$

where $\text{Neigh}(\phi)$ represents the set of unmatched nodes that are directly connected to the nodes in the current partial mapping ϕ.

The candidate set P_ϕ is generated by considering nodes that are directly connected to the already matched nodes in ϕ. If no such connected nodes exist (*i.e.,* in the initial state where $\phi = \varnothing$), P_ϕ includes all remaining unmatched node pairs, which is defined as:

$$P_\phi = \big\{(u, v) | u \in N_p, v \in N_t\big\}. \quad (21)$$

For each candidate pair, the feasibility is determined based on the feasibility match functions in Eq. (17) and Eq. (18). If a candidate pair satisfies the conditions of the matching functions, it is immediately added to ϕ as:

$$\phi \leftarrow \phi \cup \big\{(u, v) \in P_\phi\big\}. \quad (22)$$

This process will recursively continue to expand ϕ by exploring further candidate pairs, ensuring that the addition of (u, v) to ϕ satisfies the feasibility rules and maintains a valid subgraph isomorphism.

Feasibility Functions: Two feasibility rules are considered for both nodes and edges. The node feasibility function, $M_{\text{node}}(u, v)$, ensures that all attributes of node v either match those of node u or are wildcard values. The edge feasibility function, $M_{\text{edge}}((u, u'), (v, v'))$, ensures that all attributes of the edge (v, v') meet or exceed the corresponding attributes of the edge (u, u'). These functions evaluate attribute alignments such as network conditions and operational capabilities, ensuring that only feasible mappings are considered.

Recursive Matching: The matching process for each candidate pair $(u, v) \in P_\phi$ proceeds only if the nodes and their edges pass the feasibility checks. If a pair (u, v) satisfies these feasibility checks, it is added to the matching solution ϕ. The algorithm then recursively expands ϕ by updating the candidate set P_ϕ to include new pairs that are directly connected to the updated mapping ϕ. The process continues until no more candidate pairs can be added, resulting in a complete and feasible subgraph isomorphism.

Backtracking: If no feasible pairs can be found for a given mapping ϕ during the recursive matching process, it backtracks to the previous mapping ϕ to explore alternative pairs. This involves removing the last added pair from ϕ and trying different candidate pairs from the updated candidate set P_ϕ. This process will continue until all nodes $u \in N_p$ are mapped or deemed unmappable, identifying where policies and network capabilities are incompatible.

Attribute Updating: Upon successfully determining that G_p is isomorphic to a subgraph of G_t without conflicts, the next critical step involves updating the attributes of G_t to reflect the policies applied from G_p. This attribute updating process is pivotal for maintaining an up-to-date representation of network capabilities and historical policies, which is essential for subsequent conflict detection with new policy graphs.

For each node and edge in G_p that corresponds to elements in G_t, the attributes for nodes can be updated by:

$$A_v[a_i] \leftarrow A_u[a_i] \ \text{ if } \ A_v[a_i] = \text{`*'}. \quad (23)$$

Similarly, the attributes for links can be updated by:

$$A_{(v,v')}[a_i] \leftarrow \begin{cases} A_{(u,u')}[a_i] & \text{if } A_{(v,v')}[a_i] = \text{`*'}, \\ A_{(v,v')}[a_i] - A_{(u,u')}[a_i] & \text{if } A_{(v,v')}[a_i] \in \mathbb{R}. \end{cases} \quad (24)$$

Authorized licensed use limited to: Amrita Vishwa Vidyapeetham Chennai Campus. Downloaded on January 03,2026 at 12:35:01 UTC from IEEE Xplore. Restrictions apply.

V. Experimental Results

In this section, we evaluate the performance of the proposed GIA framework to verify its effectiveness in generalizing evolving user intents without a predefined template. All experiments are implemented on a Ubuntu desktop equipped with 64GB RAM and an Intel(R) Core(TM) i7-9700K CPU operating at 3.6GHz, using Python 3.11.

A. Experimental Setup

1) Overview: We implement and evaluate two aspects of the GIA framework of intent translation and negotiation.

- *GIA Generator* The GIA generator is designed to convert natural language directives into adaptable graph abstractions. We test this component by introducing a diverse set of intents, ranging from easy to complex and even previously unseen scenarios, to assess how well the proposed framework interprets and translates these intents.
- *Intent Negotiator* The intent negotiator identifies potential conflicts between the policy graph and the deployed policy graph. To evaluate the intent negotiation capability, we introduce policy conflicts into the network setup to test the intent accepted/rejected rate and its detection responsiveness.

2) LLMs Selection: To evaluate the GIA framework's performance across models with varying capabilities, we selected three LLMs from OpenAI: GPT-3.5, GPT-4, and GPT-4o (omni). These models represent successive advancements in natural language understanding and code generation, which enables a thorough evaluation of the GIA framework by controlling for consistent input across different models and observing its performance under varied model complexities. GPT-3.5 serves as a reference model to evaluate the improvements in handling complex scenarios, while GPT-4, with its enhanced architecture, was used to test the framework's capability in handling more intricate code generation tasks. Finally, GPT-4o optimizes both response speed and computational efficiency, making it suitable for high-demand code generation scenarios.

3) Comparison Baseline: To evaluate the proposed GIA framework, we implemented a baseline approach inspired by [22], which uses code generation for network management. Although their method does not specifically address intent translation, we adapted it as a baseline without knowledge enhancement, while the GIA framework incorporates this feature. This comparison isolates the impact of knowledge-enhanced prompts on adaptability, highlighting the GIA framework's ability to handle diverse and complex intent types. Additionally, while NER-based methods are commonly used for intent translation, they rely on predefined intent templates and fixed intent types, which greatly limits their adaptability to a narrow range of intents. In contrast, our GIA framework treats intent translation as a code-generation task, enabling it to handle a broader variety of intent types. Therefore, NER methods are not suitable for comparison in this study.

B. Dataset and Evaluation Metrics

1) Network Intent Dataset: To validate the effectiveness and adaptability of the proposed GIA framework, we developed a Network Intent Dataset (NID) that categorizes network intents into four levels: Easy, Medium, Hard, and Unseen. The Easy-level intents involve operations on a single switch or host with basic operations. Medium-level intents require interaction among multiple switches or hosts, while Hard-level intents cover domain-level operations, demanding a deeper understanding of the network. To evaluate the GIA framework's capability for handling novel scenarios, we introduced an "Unseen Intents" category that includes attributes or operations that are not included in the knowledge-enhanced prompts. Each category of intents is exemplified in Table III.

Additionally, to simulate the dynamic nature of users' specified intents, the dataset includes various entities (switches, hosts, domains) and network functions such as sFlow, Deep Packet Inspection (DPI), and NetFlow, among others. Attributes like timing, bandwidth, and latency are incorporated, providing a broad testing basis for evaluating the GIA framework under diverse network requirements.

2) Golden Answer Selector: We have developed "golden answers" for each intent in the NID dataset with the help of human experts. These answers are stored as Networkx graphs in JSON format and serve as the ground truth for assessing the quality of the generated GIA graphs.

3) Evaluation Metrics: To evaluate our framework's performance, We compare the generated graphs against the golden answers using several evaluation metrics. Let S_T and S_P represent the sets of nodes, edges, and their respective attributes in the label (ground truth) and generated graphs, respectively. For each intent sample *i*, we denote the ground truth graph as $S_{T_i} = (N_{T_i}, E_{T_i}, A_{N_{T_i}}, A_{E_{T_i}})$ and the generated graph as $S_{P_i} = (N_{P_i}, E_{P_i}, A_{N_{P_i}}, A_{E_{P_i}})$. Specifically, we employ "Precision" to measure the framework's capability to generate Python snippets, which is defined as:

$$\text{Precision}_{\text{avg}} = \frac{1}{4n}\sum_{i=1}^{n}\left(\frac{\left|N_{T_i}\cap N_{P_i}\right|}{\left|N_{P_i}\right|} + \frac{\left|E_{T_i}\cap E_{P_i}\right|}{\left|E_{P_i}\right|} + \frac{\left|A_{N_{T_i}}\cap A_{N_{P_i}}\right|}{\left|A_{N_{P_i}}\right|} + \frac{\left|A_{E_{T_i}}\cap A_{E_{P_i}}\right|}{\left|A_{E_{P_i}}\right|}\right). \tag{25}$$

To assess the framework's ability to accurately identify the correct elements, we utilize "Recall", which is calculated as:

$$\text{Recall}_{\text{avg}} = \frac{1}{4n}\sum_{i=1}^{n}\left(\frac{\left|N_{T_i}\cap N_{P_i}\right|}{\left|N_{T_i}\right|} + \frac{\left|E_{T_i}\cap E_{P_i}\right|}{\left|E_{T_i}\right|} + \frac{\left|A_{N_{T_i}}\cap A_{N_{P_i}}\right|}{\left|A_{N_{T_i}}\right|} + \frac{\left|A_{E_{T_i}}\cap A_{E_{P_i}}\right|}{\left|A_{E_{T_i}}\right|}\right). \tag{26}$$

To facilitate the analysis of the framework's overall performance, we employ the "F1 Score", which is calculated as:

$$\text{F1} = \frac{2\times\text{Precision}_{\text{avg}}\times\text{Recall}_{\text{avg}}}{\text{Precision}_{\text{avg}}+\text{Recall}_{\text{avg}}}. \tag{27}$$

Authorized licensed use limited to: Amrita Vishwa Vidyapeetham Chennai Campus. Downloaded on January 03,2026 at 12:35:01 UTC from IEEE Xplore. Restrictions apply.

TABLE III
DESCRIPTION OF NETWORK INTENT DATASET WITH THREE COMPLEXITY LEVELS AND AN UNSEEN INTENT DATASET

Intent Datasets	# of intents	Description	Examples
Easy-level	20	Operate on a node (switch/host) with a basic function	"Add a firewall to switch S1"
Medium-level	20	Operate 2 switches/hosts with at least 1 function	"Migrate VM1 from host h6 to host h7"
Hard-level	20	Domain-level operations with Medium functions	"Restart the switches in HR Department now"
Unseen Intents	20	Intent Nodes (switch/host) and functions do not exist in the knowledge-enhanced prompt.	

TABLE IV
EVALUATION SUMMARY FOR THREE LLMS WITH/WITHOUT KNOWLEDGE-ENHANCEMENT PROMPTING

	without Knowledge			**with Knowledge**		
Easy-level	Precision	Recall	F1	Precision	Recall	F1
GPT-3.5	0.831	0.838	0.833	0.898	0.938	0.903
GPT-4	0.838	0.838	0.838	**1**	**1**	**1**
GPT-4o	0.850	0.850	0.850	**1**	**1**	**1**
Medium-level	Precision	Recall	F1	Precision	Recall	F1
GPT-3.5	0.713	0.717	0.714	0.774	0.781	0.773
GPT-4	0.727	0.750	0.736	**0.960**	**0.975**	**0.966**
GPT-4o	0.796	0.8	0.798	**0.960**	**0.975**	**0.966**
Hard-level	Precision	Recall	F1	Precision	Recall	F1
GPT-3.5	0.578	0.608	0.584	0.723	0.770	0.730
GPT-4	0.663	0.662	0.662	**0.938**	0.931	**0.934**
GPT-4o	0.626	0.627	0.624	0.933	**0.934**	0.932

To evaluate the exact match between the generated graph and the baseline graph, we apply "Accuracy", which measures the proportion of samples where the generated graph exactly matches its label graph. Accuracy is defined as:

$$\text{Accuracy} = \frac{1}{n}\sum_{i=1}^{n}\delta\big(S_{T_i}, S_{P_i}\big), \tag{28}$$

where $\delta(S_{T_i}, S_{P_i})$ is 1 if the ground truthS_{T_i} exactly matches the predicted graph S_{P_i}, and 0 otherwise.

To evaluate the efficiency of the model in utilizing computational resources, we use "Token Cost Efficiency", which is defined as:

$$\text{Token Cost Efficiency} = \frac{\text{F1}}{\text{Tokens} \times \text{Time}}. \tag{29}$$

C. Knowledge-Enhanced GIA

We conduct our study using three state-of-the-art LLMs, including GPT-3.5, GPT-4, and GPT-4o, to evaluate our framework's ability to generalize evolving user intents. To ensure that the semantics of the intent were accurately preserved in the generated GIA graph, we set the temperature parameter of each model to 0, aiming for the most deterministic output. For robustness, each experiment was repeated five times, and the reported values represent the averages of these trials. The benchmark results, which summarize the precision, recall, and F1 score of each both with and without factual knowledge, are presented in Table IV.

1) Code Quality Analysis: From the experimental results, we observe several key insights into the performance of the proposed GIA framework across varying levels of intent complexity. Firstly, GPT-4 and GPT-4o demonstrate comparable performance at the easy and medium levels of intent complexity. However, at the hard level, GPT-4 slightly outperforms GPT-4o. Specifically, when knowledge is integrated with the prompt, GPT-4 achieves an F1 score of 0.934 at the hard level, marginally higher than GPT-4o's 0.932. This difference can be attributed to the architectural optimizations of GPT-4o, which prioritize efficiency and response speed making it suitable for high-demand applications but with a minor trade-off in depth for complex reasoning tasks. As a result, GPT-4o excels in handling straightforward or moderately level intents, while GPT-4 is better suited for handling intricate intents involving layered logic and nuanced code generation.

Secondly, a consistent decline in performance metrics is observed for all three LLMs as intent complexity increases. This decline is attributed to the exacerbated issue of hallucinations for LLMs to the more complex and logical analysis required to extract operational entities and the potential inclusion of multiple sub-intents within harder-level intents. Despite these complexities, both GPT-4 and GPT-4o exhibit significant improvements with knowledge enhancement. Specifically, GPT-4o shows a nearly 49.36% increase in F1 score at the hard level, compared to results without knowledge enhancement, while GPT-4 shows a 41.09% increase. The reason GPT-4o's improvement exceeds that of GPT-4 by 7% is because the baseline performance of GPT-4o without knowledge is much lower. However, when both models are enhanced with knowledge, GPT-4 slightly outperforms GPT-4o, demonstrating its inherently stronger baseline performance and better capability to leverage knowledge integration for enhancing its reasoning capabilities. This aligns with our earlier analysis that GPT-4, even with knowledge enhancement, tends to perform slightly better than GPT-4o in handling complex intents.

Finally, the integration of factual knowledge significantly boosts performance across all complexity levels for each LLM. At the easy level, the maximum improvement in F1 score due to knowledge integration is about 19.33%, while at the medium level, it rises to approximately 31.25%. These improvements highlight the critical role of factual knowledge in reducing LLM hallucinations and enhancing the model's capacity to translate high-level intents into structured graphs, thereby improving performance substantially. This experiment confirms the effectiveness of our approach.

2) Generated Graph Analysis: To evaluate the effectiveness of the generated GIA graphs, we analyze their exact

Authorized licensed use limited to: Amrita Vishwa Vidyapeetham Chennai Campus. Downloaded on January 03,2026 at 12:35:01 UTC from IEEE Xplore. Restrictions apply.

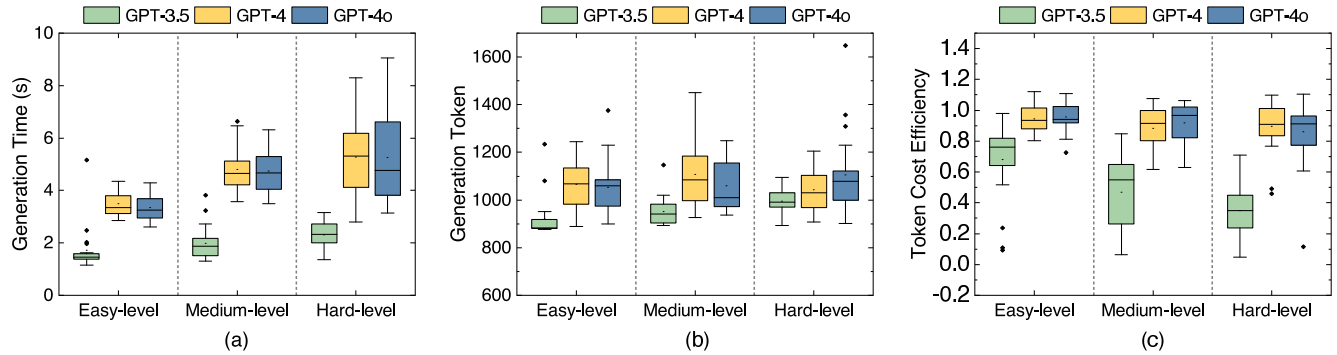


Fig. 4. Cost analysis on delay, token and token cost efficiency (TCE).

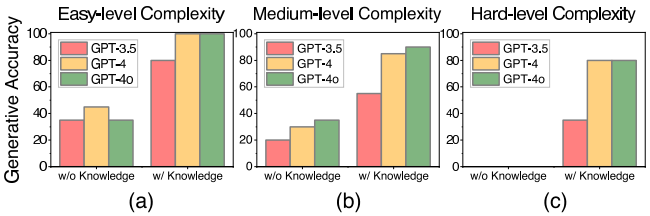


Fig. 5. Exact match accuracy comparison across three LLMs with/without knowledge-enhanced prompting.

TABLE V
ERROR TYPE SUMMARY OF LLMS GENERATED CODE

LLM's error type (GPT-4o)	Easy-level	Medium-level	Hard-level
No. Nodes & Links & Attr.	40	91	118
Syntax error	0	0	0
Imaginary/lack graph nodes	0	1	4
Imaginary/lack node attributes	0	1	11
Imaginary/lack graph links	0	1	6
Imaginary/lack link attributes	0	0	6

match accuracy across easy, medium, and hard levels of intent complexity. Specifically, we compare the accuracy of GIA graph generation using methods with and without knowledge-enhanced prompts. As illustrated in Fig. 5, our method consistently outperforms the baseline across all three levels of intent complexity. Notably, both the proposed method and the baseline exhibited a decline in accuracy as the complexity of the intents increased. For intents at the easy level, both GPT-4 and GPT-4o achieved the perfect generation of accurate GIA graphs. These results highlight the effectiveness of our approach in mitigating the hallucination issues, ensuring that the generated graphs precisely mirror the ground truth, particularly when the specified intents are straightforward. For medium complexity intents, we observe improvements in exact match accuracy of 175%, 183.3%, and 157.1% for the three LLMs—GPT-3.5, GPT-4, and GPT-4o—compared to the baseline method. These significant enhancements demonstrate that incorporating network contextual knowledge not only helps in accurately capturing the structured relationships within the intents but also greatly improves the models' ability to handle more nuanced details. For intents at the hard level, although the baseline method struggled to correctly identify any declared intents, our proposed method maintained an accuracy of 80% on both GPT-4 and GPT-4o.

3) Code Error Analysis: As illustrated in Table V, the analysis of error types in code generated by GPT-4o reveals notable trends. Notably, there were no syntax errors across easy, medium, and hard levels of intent complexity, indicating robust syntax understanding by the model. However, for medium-level intents, GPT-4o exhibited three instances of errors related to the presence of imaginary nodes, node attributes, and graph links. At the hard level, the predominant category of errors, accounting for approximately 62.9% of all mistakes, involved imaginary attributes of nodes and links. These errors were more frequent and diverse, highlighting challenges in accurately generating more complex graph structures.

4) Cost Analysis: By analyzing Fig. 4, we observe the performance of GPT-3.5, GPT-4, and GPT-4o in handling intents at different complexity levels. Across all charts, as the intent level increases from easy to hard, notable trends emerge in the model's generation time, token count, and token cost efficiency. Specifically, all LLMs display an increase in generation time and token count, while token cost efficiency decreases correspondingly. This trend indicates that intents of higher complexity require increased time and more tokens to complete. Interestingly, GPT-3.5 consistently achieves the lowest generation time and token usage. However, as the complexity of the intent increases, particularly at the hard level, its token cost efficiency significantly decreases due to the model's weaker reasoning capabilities. In contrast, GPT-4 and GPT-4o demonstrate superior performance, especially in handling complex intents. As depicted in Fig. 4(b) and Fig. 4(c), although GPT-4o has faster generation speeds, GPT-4 exhibits stronger reasoning abilities and higher token cost efficiency under challenging intents. Compared to GPT-3.5, these two models achieve shorter generation times, lower token counts, and higher token cost efficiency.

D. Adaptability Analysis

To validate the flexibility and adaptability of the proposed GIA framework, we further conduct a comparative analysis on the average F1 score and accuracy of generated code and GIA graphs across unseen intents. This assessment ensures that the entities, edges, and attributes specified in the user-defined intents have never appeared in the enhanced prompts,

Authorized licensed use limited to: Amrita Vishwa Vidyapeetham Chennai Campus. Downloaded on January 03,2026 at 12:35:01 UTC from IEEE Xplore. Restrictions apply.

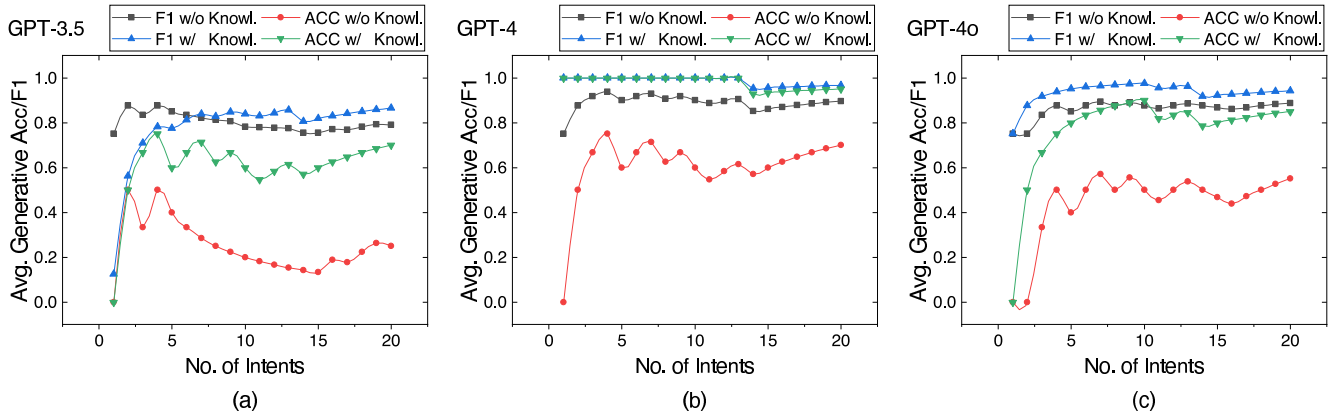


Fig. 6. Adaptability analysis on average accuracy and F1 score for unseen intents. ('w/o' denotes 'without', and 'w/' denotes 'with').

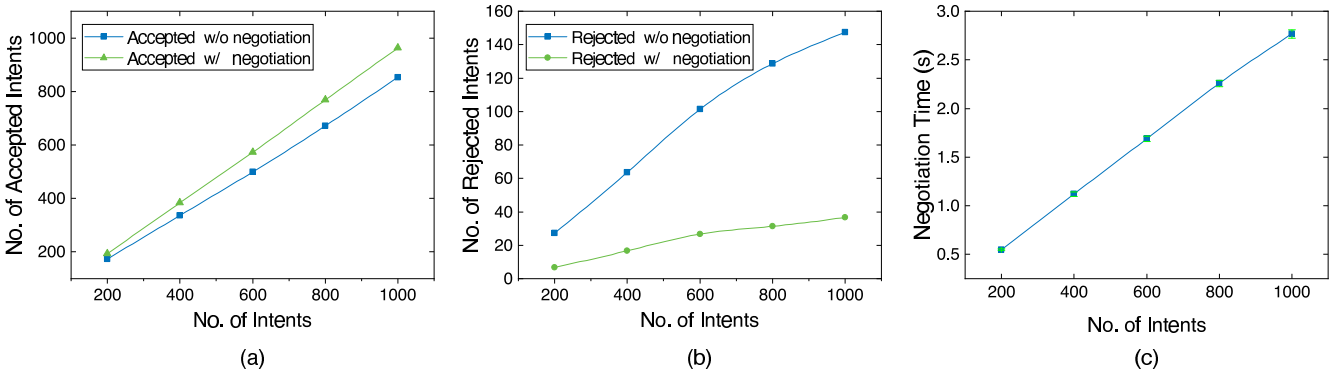


Fig. 7. Experiment results of the graph-based intent negotiation.

simulating real-world scenarios where unforeseen intent must be handled.

As depicted in Fig. 6, the results clearly demonstrate that the F1 scores and the accuracy rate are generally higher with knowledge enhancement across all models. This underscores the impact of knowledge-enhanced prompts in improving model performance on unseen intents. Particularly, knowledge-enhanced conditions consistently outperform no-enhanced setups, highlighting the adaptability of the proposed GIA framework. Comparing the models, GPT-3.5 exhibits the most fluctuation, especially in the non-enhanced scenario where both F1 scores and accuracy rates are notably getting lower. In comparison, GPT-4 maintains high stability in performance with knowledge enhancement, almost consistently exhibiting high F1 scores and accuracy rates across all intents, especially in the first 13 intents. GPT-4o, while generally performing better with knowledge enhancement than without, shows slightly less consistency compared to GPT-4.

Furthermore, a notable observation is the performance on specific intents—such as the first and thirteenth intents—where all LLMs exhibit a performance drop in accuracy. This indicates that these intents may be inherently more challenging or that models encounter particular difficulties when handling certain types of unseen intents. The experimental results underscore the importance of knowledge enhancement in improving the performance of LLMs on previously never-seen intents. These results demonstrate the effectiveness of the proposed GIA framework in handling novel intents in future applications.

E. Intent Conflict Verification and Negotiation

To evaluate our graph-based intent negotiation approach, we conducted a simulation-based experiment using a fixed SDN topology. The topology consists of 9 switches interconnected in a mesh configuration, each connected to a host. This setup supports multiple paths between any two hosts, providing flexible forwarding options to meet various intent demands.

In this experiment, we generated up to 1000 forwarding intents by randomly selecting pairs of hosts. Each intent was assigned a random QoS requirement, including bandwidth and latency. To simulate dynamic network conditions, each link's available bandwidth and real-time latency were randomly updated before processing each intent. The Intent Negotiator then checked each intent against the current network state. If a conflict was detected (i.e., the intent could not be supported due to resource constraints), the Intent Negotiator attempted to resolve it by selecting alternative paths and re-evaluating compatibility.

This experimental setup allowed us to measure three key aspects: 1) the number of intents accepted, 2) the number of intents rejected, and 3) the cumulative processing overheads for intent negotiation (average time taken). We conducted five simulation rounds with intent counts incrementally ranging from 200 to 1000.

Authorized licensed use limited to: Amrita Vishwa Vidyapeetham Chennai Campus. Downloaded on January 03,2026 at 12:35:01 UTC from IEEE Xplore. Restrictions apply.

As shown in Fig. 7(a), the number of intents accepted with negotiation consistently surpassed those without negotiation, showing an average increase of 13.68%. This improvement underscores the effectiveness of our graph-based method for intent negotiation, which resolves conflicts not only between current intent and historical intents but also with the underlying network state. Fig. 7(b) illustrates the number of intents rejected with and without intent negotiation. As the number of intents increases from 200 to 1000, the experimental results clearly show fewer rejections with negotiation than without. Specifically, the average rejection rate for intents processed without negotiation is increased by 296.9% higher than those with negotiation. Additionally, Fig. 7(c) presents the negotiation processing overheads. We note that the average time for intent negotiation shows a linear increase as the number of intents increases. Despite this increase, the processing times remain well within industry standards, highlighting the utility of employing subgraph isomorphism for robust intent conflict resolution.

F. Discussion

In this subsection, we analyze the computational complexity and scalability of the proposed GIA framework and discuss potential directions for future work.

1) Computational Complexity: The computational complexity of the GIA framework stems from its two components:

- *GIA Generator:* The GIA Generator component leverages the generative pre-trained transformers (GPTs) API to translate high-level intents into adaptable graphs. The computational complexity of the GPTs model, which consists of a stack of transformers, relies on the input context length. As an autoregressive model, GPTs generate each token based on all the input context lengths, leading to a computational complexity of approximately $O(n^2 \cdot d)$ due to the multi-head attention mechanism, where *n* is the input context and *d* is the model's hidden layer size. The feed-forward network contributes an additional $O(n \cdot d^2)$. Thus, the overall computational complexity of the GIA generator can be expressed as $O(n^2 \cdot d + n \cdot d^2)$ [39].
- *Intent Negotiator:* The VF2 algorithm, employed in our intent negotiator, has a worst-case computational complexity of $O(N! \cdot N)$ [38], where *N* is the number of nodes in the pattern graph G_p (the intent policy graph). This factorial growth results from exploring all possible node mappings between the pattern graph (representing the user's intent) and the target graph (representing the deployed policy). However, VF2's efficiency is significantly enhanced by feasibility rules that prune infeasible mappings early in the search process, effectively reducing the search space. Additionally, since the pattern graph, which represents the user's intent, typically has far fewer nodes than the target graph (which represents the network's infrastructure), the search space is further limited, making the algorithm efficient in practice.

2) Scalability Analysis: The computational complexity of GPTs is dominated by the self-attention mechanism, which scales quadratically with the input context length *n*. The overall complexity can be approximated as $O(n^2 \cdot d)$, where *n* is the sum of the shared knowledge context n_k and the user's intents n_i. When processing a large number of intents *m*, the total input context length becomes $n = n_k + m \cdot n_i$. By batching *m* intents into a single API call, the length of the user intents n_i scales proportionally, while avoiding the repeated processing of the shared knowledge context n_k. This batching approach primarily affects the self-attention complexity, resulting in an overall complexity of $O((n_k + m \cdot n_i)^2 \cdot d)$. As *m* increases, the term $m \cdot n_i$ dominates, and the complexity can be approximated as $O(m^2 \cdot n_i^2 \cdot d)$, assuming that the shared knowledge context n_k is relatively small compared to the total length of user intents.

However, batching too many intents into a single API call may lead to issues such as context overlap and hallucination risks, which can degrade the quality of the model's outputs. To mitigate these challenges, we recommend processing each intent separately using parallel API calls. This approach eliminates the risk of overlapping context, though it introduces additional overhead from the API calls themselves. In terms of computational complexity, processing each intent independently results in an overall complexity of $O(n_k^2 \cdot d) + m \cdot O(n_i^2 \cdot d)$. This ensures that the input lengths remain manageable, maintaining both scalability and responsiveness.

3) Future Work: To further enhance scalability and reduce computational complexity, future improvements will explore the integration of parallel API calls and retrieval-augmented generation (RAG) techniques. RAG will dynamically retrieve only the most relevant knowledge for each intent, rather than providing the entire knowledge base. This targeted retrieval approach aims to minimize input redundancy and improve both processing efficiency and output quality.

VI. Conclusion

This paper presented the GIA framework, which utilizes LLMs to translate high-level user intents into structured graphs, thus replacing rigid template-based methods and enabling greater adaptability in intent-driven networks. We addressed the issue of hallucination in LLMs with a knowledge-enhanced prompting technique that improves code quality. We also reformulated intent negotiation by framing it as a subgraph isomorphism problem, ensuring that intents align with the network's underlying states and avoid conflicts with existing policies. Simulation results further confirmed the effectiveness of our SIP-based intent negotiation method. GIA not only demonstrates the capability to dynamically adapt to network changes and evolving requirements while maintaining high accuracy in intent translation but also significantly reduces hallucinations in code generation and enhances the adaptability of intent translation, outperforming non-knowledge-augmented methods by over 30% for seen intents and 180% for unseen intents in terms of the F1 score.

References

[1] A. El-Hassany, P. Tsankov, L. Vanbever, and M. Vechev, "Network-wide configuration synthesis," in *Proc. Int. Conf. Comput.-Aided Verification (CAV)*, 2017, pp. 261–281.

Authorized licensed use limited to: Amrita Vishwa Vidyapeetham Chennai Campus. Downloaded on January 03,2026 at 12:35:01 UTC from IEEE Xplore. Restrictions apply.

[2] K. Mehmood, K. Kralevska, and D. Palma. "Knowledge-driven intent life-cycle management." 2024. [Online]. Available: https://doi.org/10.36227/techrxiv.171986543.36228657/v2
[3] A. Boutouchent et al., "AMANOS: An intent-driven management and orchestration system for next-generation cloud-native networks," *IEEE Commun. Mag.*, vol. 62, no. 6, pp. 42–49, Jun. 2024.
[4] A. S. Jacobs, R. J. Pfitscher, R. A. Ferreira, and L. Z. Granville, "Refining network intents for self-driving networks," in *Proc. Afternoon Workshop Self-Driving Netw.*, 2018, pp. 15–21.
[5] M.-T.-A. Nguyen, S. B. Souihi, H.-A. Tran, and S. Souihi, "When NLP meets SDN: An application to global Internet exchange network," in *Proc. IEEE Int. Conf. Commun. (ICC)*, 2022, pp. 2972–2977.
[6] B. Tian et al., "Safely and automatically updating in-network ACL configurations with intent language," in *Proc. ACM Special Interest Group Data Commun. (SIGCOMM)*, 2019, pp. 214–226.
[7] R. Soulé et al., "MERLIN: A language for managing network resources," *IEEE/ACM Trans. Netw.*, vol. 26, no. 5, pp. 2188–2201, Oct. 2018.
[8] A. Abhashkumar, J.-M. Kang, S. Banerjee, A. Akella, Y. Zhang, and W. Wu, "Supporting diverse dynamic intent-based policies using janus," in *Proc. Int. Conf. Emerg. Netw. Exp. Technol. (CoNEXT)*, 2017, pp. 296–309.
[9] C. Prakash et al., "PGA: Using graphs to express and automatically reconcile network policies," in *Proc. ACM SIGCOMM*, 2015, pp. 29–42.
[10] H. Kim, J. Reich, A. Gupta, M. Shahbaz, N. Feamster, and R. Clark, "Kinetic: Verifiable dynamic network control," in *Proc. USENIX Symp. Netw. Syst. Design Implement. (NSDI)*, 2015, pp. 59–72.
[11] R. H. Ribeiro et al., "A bottom-up approach for extracting network intents," in *Proc. Int. Conf. Adv. Inf. Netw. Appl. (AINA)*, 2020, pp. 858–870.
[12] K. Dzeparoska, A. Tizghadam, and A. Leon-Garcia, "Emergence: An intent fulfillment system," *IEEE Commun. Mag.*, vol. 62, no. 6, pp. 36–41, Jun. 2024.
[13] K. Dzeparoska, J. Lin, A. Tizghadam, and A. Leon-Garcia, "LLM-based policy generation for intent-based management of applications," in *Proc. Int. Conf. Netw. Service Manag. (CNSM)*, 2023, pp. 1–7.
[14] K. Dzeparoska, A. Tizghadam, and A. Leon-Garcia. "Intent assurance using LLMs guided by intent drift." 2024. [Online]. Available: https://arxiv.org/abs/2402.00715.
[15] C. Wang, M. Scazzariello, A. Farshin, S. Ferlin, D. Kostić, and M. Chiesa, "NetConfEval: Can LLMs facilitate network configuration?" in *Proc. ACM Netw.*, vol. 2, 2024, pp. 1–25.
[16] X. Guan et al., "Mitigating large language model hallucinations via autonomous knowledge graph-based retrofitting," in *Proc. AAAI Conf. Artif. Intell.*, vol. 38, 2024, pp. 18126–18134.
[17] W. Sun, Z. Shi, S. Gao, P. Ren, M. de Rijke, and Z. Ren, "Contrastive learning reduces hallucination in conversations," in *Proc. AAAI Conf. Artif. Intell.*, vol. 37, 2023, pp. 13618–13626.
[18] S. Dhuliawala et al. "Chain-of-verification reduces hallucination in large language models." 2023. [Online]. Available: https://arxiv.org/abs/2309.11495.
[19] C. Wang, M. Scazzariello, A. Farshin, D. Kostić, and M. Chiesa. "Making network configuration human friendly." 2023. [Online]. Available: https://arxiv.org/abs/2309.06342.
[20] Y. Li et al., "Competition-level code generation with AlphaCode," *Science*, vol. 378, no. 6624, pp. 1092–1097, 2022.
[21] A. M. Dakhel et al., "Github copilot AI pair programmer: Asset or liability?" *J. Syst. Softw.*, vol. 203, Sep. 2023, Art. no. 111734.
[22] S. K. Mani et al., "Enhancing network management using code generated by large language models," in *Proc. ACM Workshop Hot Topics Netw. (HotNets)*, 2023, pp. 196–204.
[23] A. S. Jacobs et al., "Hey, LUMI! Using natural language for intent-based network management," in *Proc. USENIX Annu. Tech. Conf. (USENIX ATC)*, 2021, pp. 625–639.
[24] C. H. Cesila et al., "Chat-IBN-RASA: Building an intent translator for packet-optical networks based on RASA," in *Proc. IEEE Int. Conf. Netw. Softw. (NetSoft)*, 2023, pp. 534–538.
[25] J. Lin, K. Dzeparoska, A. Tizghadam, and A. Leon-Garcia, "AppleSeed: Intent-based multi-domain infrastructure management via few-shot learning," in *Proc. IEEE Int. Conf. Netw. Softw. (NetSoft)*, 2023, pp. 539–544.
[26] J. Wang et al., "Network meets ChatGPT: Intent autonomous management, control, and operation," *J. Commun. Inf. Netw.*, vol. 8, no. 3, pp. 239–255, 2023.
[27] D. M. Manias, A. Chouman, and A. Shami, "Towards intent-based network management: Large language models for intent extraction in 5G core networks," in *Proc. Int. Conf. Design Rel. Commun. Netw. (DRCN)*, 2024, pp. 1–6.
[28] T. Brown et al., "Language models are few-shot learners," in *Proc. Adv. Neural Inf. Process. Syst.*, vol. 33, 2020, pp. 1877–1901.
[29] Z. Ji et al., "Survey of hallucination in natural language generation," *ACM Comput. Surveys*, vol. 55, no. 12, pp. 1–38, 2023.
[30] J. Baek, A. F. Aji, and A. Saffari, "Knowledge-augmented language model prompting for zero-shot knowledge graph question answering," 2023, *arXiv:2306.04136*.
[31] Z. Ji et al., "RHO (ρ): Reducing hallucination in open-domain dialogues with knowledge grounding," 2022, *arXiv:2212.01588*.
[32] A. Agarwal, S. Gawade, A. A. Prakash, and P. Bhattacharyya, "KITLM: Domain-specific knowledge integration into language models for question answering," in *Proc. Int. Conf. Nat. Lang. Process. (ICON)*, 2023, pp. 202–294.
[33] J. Jiang, K. Zhou, Z. Dong, K. Ye, W. X. Zhao, and J.-R. Wen, "StructGPT: A general framework for large language models to reason over structured data," 2023, *arXiv:2305.09645*.
[34] S. Zampelli, Y. Deville, and C. Solnon, "Solving subgraph isomorphism problems with constraint programming," *Constraints*, vol. 15, pp. 327–353, Jul. 2010.
[35] NetworkX Developers. "NetworkX: Network analysis in Python." 2024. [Online]. Available: https://networkx.org/
[36] J. Maynez, S. Narayan, B. Bohnet, and R. McDonald, "On faithfulness and factuality in abstractive summarization," 2020, *arXiv:2005.00661*.
[37] S. Bubeck et al., "Sparks of artificial general intelligence: Early experiments with GPT-4," 2023, *arXiv:2303.12712*.
[38] L. P. Cordella, P. Foggia, C. Sansone, and M. Vento, "A (sub)graph isomorphism algorithm for matching large graphs," *IEEE Trans. Pattern Anal. Mach. Intell.*, vol. 26, no. 10, pp. 1367–1372, Oct. 2004.
[39] L. Kaiser. "Tensor2Tensor transformers: New deep models for NLP." 2021. [Online]. Available: https://nlp.stanford.edu/seminar/details/lkaiser.pdf



Shiwen Kou received the B.Eng. degree in electrical and information engineering from Northwest Minzu University, Lanzhou, China, in 2019. He is currently pursuing the Ph.D. degree in information and communication with Xidian University, Xi'an, China. His research interests include intent-based networking, software-defined networking, large language models, machine learning, and deep learning.



Chungang Yang (Senior Member, IEEE) is a Full Professor with Xidian University, where he leads the GUIDE Research Team of Game, Utility, and Artificial Intelligent Design for Emerging Communications. His research interests include artificially intelligent 6G wireless mobile networks, intent-driven networks, space–terrestrial networks, and game theory for emerging communication networks.



Mohan Gurusamy (Senior Member, IEEE) received the Ph.D. degree in computer science and engineering from the Indian Institute of Technology Madras, Chennai, India, in 2000. In 2000, he joined the National University of Singapore, Singapore, where he is currently an Associate Professor with the Department of Electrical and Computer Engineering. He has about 220 publications to his credit, including two books and three book chapters in the area of optical networks. His research experience and interests are in the areas of Internet of Things, 5G networks, software-defined networks, network function virtualization, cloud computing, data center networks, and optical networks. He was a TPC co-chair for several conferences and was an Editor for IEEE TRANSACTIONS ON CLOUD COMPUTING and is serving on the editorial board for *Computer Networks* (Elsevier) and *Photonic Network Communications* (Springer).

Authorized licensed use limited to: Amrita Vishwa Vidyapeetham Chennai Campus. Downloaded on January 03,2026 at 12:35:01 UTC from IEEE Xplore. Restrictions apply.